DDR3 DRAM Controller — Complete Project Analysis

1. PROJECT OVERVIEW

This project implements a **complete DDR3 DRAM Memory Controller** in Verilog/SystemVerilog RTL, designed as a custom ASIC IP block for Axiado Corporation's security SoC. It bridges a processor/SoC system bus to physical DDR3 DRAM modules, handling every aspect of DDR3 operation including power-up initialization, PHY timing calibration, and high-speed read/write operations.

What this project does in one sentence:
It translates simple 512-bit read/write commands from a CPUinto the precise, multi-step DDR3 electrical protocol required to transfer data to/from physical DRAMchips, then returns the result — handlingall timing, calibration, and physical signal generation along the way.

---

2. PROJECT STRUCTURE — 5 MAJOR BLOCKS

```
DDR3 DRAM Controller/
├── DDR3MC/              ← Core DDR3 Memory Controller (ASIC)
│   ├── rtl/             ← 40+ Verilog/SV source files
│   ├── include/         ← 4 parameter header files
│   ├── sim/             ← Testbenches + DDR3 DRAM model
│   ├── const/           ← FPGA pin constraint files (.xdc)
│   └── scripts/         ← Vivado TCL build scripts
│
├── MemCtrl/             ← System-level wrapper + arbiter
│   ├── rtl/             ← Top level, arbiter, interface bridge
│   └── sim/             ← System-level testbenches
│
├── MemCtrl1/            ← Fully integrated system
│   └── src/
│       ├── import/ddr3mc/   ← DDR3MC as submodule
│       ├── import/sfc/      ← Serial Flash Controller
```

```
|       ├── import/spdc/        ← SPD Decoder Controller
|       ├── import/TMB_new/     ← Test Memory Bus (PCIe debug)
|       ├── import/scrambler/ ← 512-bit LFSR data scrambler
|       ├── import/pipeline/  ← Pipeline register utility
|       └── import/reset_sync/← Reset synchronizer
|
├── SFC/              ← Serial Flash Controller (standalone)
└── SPDC/             ← SPD Decoder Controller (standalone)
```

---

3. INCLUDE FILES — THE FOUNDATION

All four header files are ` included by every RTL module and define the entire system.

`ddr3mc_user_data.vh` — Master Configuration

```
Clock period: 5000ps → 200 MHz half data rate → DDR3-400
DIMM0 connected: YES
DIMM1, DIMM2: NOT connected
DIMM0 ranks: 2
DIC  (Drive Impedance): 2'b01 → RZQ/7
RTT  (On-Die Termination): 3'b011 → RZQ/6
RTT_WR: 2'b00 → Disabled
```

Command opcodes defined:

- `CMD_OPCODE_WRITE = 4'b1001`
- `CMD_OPCODE_READ  = 4'b1010`
- `CMD_OPCODE_NOP   = 4'b0111`
- `CMD_OPCODE_REF   = 4'b1110` (refresh)
- `CMD_OPCODE_MRS   = 4'b1100` (mode register set)
- `CMD_OPCODE_CFG   = 4'b1101` (configuration)

`ddr3mc_timing_parameters.vh` — JEDEC Timing (clock=5000ps)

Parameter	Value	Meaning
tRCD	15000/5000 = 3 cycles	RAS-to-CAS delay
tCL	6 cycles	CAS latency (read to data)
tCWL	5 cycles	CAS write latency

Parameter	Value	Meaning
tRP	3 cycles	Row precharge time
tWTR	max(4, 1) = 4 cycles	Write-to-read turnaround
tRFC	70 cycles	Refresh cycle time
tREFI	1560 cycles	Refresh interval (7.8 μs)
tZQCL	512 cycles	ZQ calibration time
t200US	40 cycles (sim)	Power-up wait
t500US	100 cycles (sim)	CKE stabilization

`ddr3mc_size_parameters.vh` — Address Geometry

```
CFG_NUM_DIMMS = 1
CFG_NUM_RANKS = 2   (1 DIMM × 2 ranks)
CFG_BANK_WIDTH = 3   (8 banks)
CFG_ROW_WIDTH = 16  (64K rows)
CFG_COL_WIDTH = 11  (2K columns)
SYS_DATA_WIDTH = 512 bits (64 bytes)
SYS_ADDR_WIDTH = 33 bits (8 GB)
DDR3_DQ_WIDTH = 64 bits
DDR3_DQS_WIDTH = 8
CMDQ_DEPTH   = 4   (4 outstanding commands)
```

`ddr3mc_phy_parameters.vh` — PHYDelay Configuration

Defines delay tap parameters, DLL configuration values, and IO cell control settings for the custom GUC ASIC PHY.

---

4. DDR3MC — CORE CONTROLLER (Detailed File-by-File)

4.1 `ddr3mc_top.v` — Absolute Top Module (715 lines)

Role: Instantiates all sub-modules and routes every internal wire between them.

External interfaces exposed:

- **System Interface** — `sys_clk`, `sys_reset_n`, `sys_cmd[3:0]`, `sys_address[32:0]`, `sys_write_data[511:0]`, `sys_write_data_mask[63:0]`, `sys_read_data[511:0]`,

`sys_read_data_valid`, `sys_read_data_pull`, `sys_cfg_wr_en/addr/data`

- **DDR3 Physical Pins —** `ddr3_A[15:0]`, `ddr3_BA[2:0]`, `ddr3_CK0/1_p/n`, `ddr3_CKE0/1`, `ddr3_CS0/1_n`, `ddr3_RAS/CAS/WE_n`, `ddr3_ODT0/1`, `ddr3_RESET_n`, `ddr3_DQ[63:0]` (inout), `ddr3_DQS_p/n[7:0]` (inout), `ddr3_DM[7:0]`, `ddr3_EVENT_n`

Critical data routing in top.v:

```verilog
// After calibration done, switch DQ/DQS source from CAL engine to PROC engine:
assign dq_output_enable  = sys_status[CAL_DONE] ? !write_en_proc : dq_output_enable
assign dqs_output_enable = sys_status[CAL_DONE] ? !write_en_proc : dqs_output_enabl
assign dqs_i1 = sys_status[CAL_DONE] ? dqs_i1_proc : dqs_i1_cal;
assign dq_i1  = sys_status[CAL_DONE] ? dq_i1_proc  : dq_i1_cal;

// DQS i1/i2 are SWAPPED to support write calibration (documented in code)
.dqs_i1(dqs_i2),   // Inverted to support write calibration
.dqs_i2(dqs_i1),   // Inverted to support write calibration
```

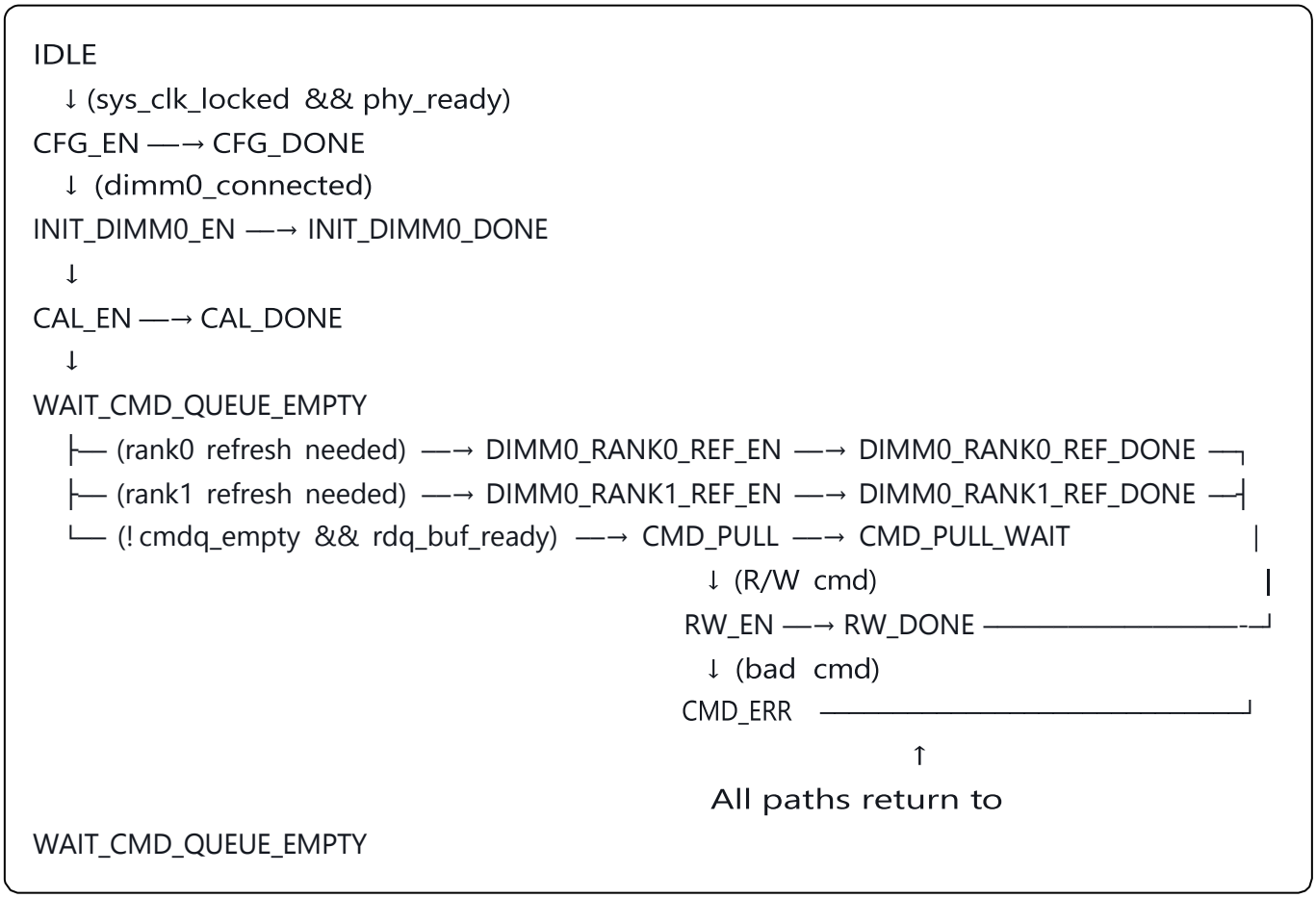
Sub-modules instantiated:

1. `ddr3mc_flow_control` — Master state machine
2. `ddr3mc_imux` — Command mux (init/cal/proc)
3. `ddr3mc_queues` — CMDQ + WDQ + RDQ FIFOs
4. `ddr3mc_init_engine` — DDR3 power-up init
5. `ddr3mc_cal_engine` — Full calibration engine
6. `ddr3mc_processing_engine` — Normal R/Woperation
7. `ddr3mc_cfg` — Configuration registers
8. `ddr3mc_phy_top` — Physical layer

4.2 `ddr3mc_flow_control.v` — Master State Machine

Role: The conductor ofthe entire controller. Sequences all phases, translates system addresses to DDR3 physical addresses, manages refresh timing.

Complete State Machine:



Address Translation (CMD_PULL_WAIT state):

```
sys_address[32:0] → split into:
  cmd_rank[0]     = bit determined by rank address range check
  cmd_bank[2:0]   = bits[bank_width+row_width+col_width-1 : row_width+col_width]
  cmd_row[15:0]   = bits[row_width+col_width-1 : col_width]
  cmd_col[10:0]   = bits[col_width-1 : 0]
```

Refresh Management:

- Two independent tREFI counters – one per rank
- `dimm0_rank0_tREFI_cnt_ok` and `dimm0_rank1_tREFI_cnt_ok` signals
- Refresh takes priority over normal R/Wcommands
- After each refresh, counter resets and counts to tREFI again

---

4.3 `ddr3mc_cfg.v` — Configuration Engine

Role: Accepts writes to configure all timing parameters and DIMM geometry. Simple 3-state FSM: IDLE → EN → DONE.

What it configures:

- All timing parameters in clock cycles: tCL, tCWL, tRCD, tCCD, tWTR, tRFC, tRP, tWR, tRTP, tWTP, tburst, tXPR, tZQCL, tMRD, tMOD, t200US, t500US, tREFI
- DIMM connection status and geometry: bank/row/col address bits, number ofranks
- DIMM address ranges: start address (sa) and end address (ea) for each DIMM and rank
- Mode register values: mr0_CL, mr1_DIC, mr1_RTT, mr1_AL, mr2_CWL, mr2_RTTWR

Outputs `cfg_err` ifan unsupported DIMM configuration is detected.

4.4 `ddr3mc_init_engine.v` — DDR3 Power-Up Initialization

Role: Executes the mandatory JEDEC DDR3 initialization sequence on every power-up. Must complete before the DRAM is usable.

Complete Initialization State Machine (25 states):

```
IDLE → ASSERT_RESET
    ↓ Hold RESET_n low
WAIT_200US (cnt = t200US_nCK = 200µs or 40 cycles in sim)
    ↓
DEASSERT_RESET → WAIT_500US (cnt = t500US_nCK = 500µs or 100 cycles in sim)
    ↓
ASSERT_CKE → CKE_HIGH_NOP → CKE_HIGH_NOP_WAIT (NOP commands while clock settles)
    ↓
WAIT_TXPR (cnt = tXPR = max(5, tRFC+10ns) cycles — exit self-refresh delay)
    ↓
MR2 → MR2_WAIT   [Program: CWL, RTT_WR into Mode Register 2]
    ↓
MR3 → MR3_WAIT   [Program: MPR (Multi-Purpose Register) mode into MR3]
    ↓
MR1 → MR1_WAIT   [Program: DIC, RTT, AL into Mode Register 1]
    ↓
MR0 → MR0_WAIT   [Program: CL, WR, DLL reset into Mode Register 0]
    ↓
ZQCL → ZQCL_WAIT (cnt = tZQCL = 512 cycles — ZQ Long Calibration)
    ↓
RANK_DONE → NEXT_RANK (repeat for each rank)
    ↓ (all ranks done)
NEXT_DIMM (repeat for each DIMM, deassert prev CKE)
    ↓
DONE
```

Mode Register Bit Fields programmed:

- **MR0:** CL (CAS latencyA6,A5,A4,A2), DLL reset A8, WR A11,A10,A9
- **MR1:** DLL enable A0, Output driver impedance A5,A1, RTT A9,A6,A2, AL A4,A3
- **MR2:** CWL A5,A4,A3, RTT_WR A10,A9
- **MR3:** All zeros (MPR disabled for normal operation)

4.5 `ddr3mc_rw_control.v` — Read/Write Command Sequencer

Role: Translates a single read or write command into the exact multi-step DDR3 command sequence. Two parallel state machines run simultaneously.

Main State Machine (State1 — DDR3 Commands):

```
IDLE → ST_CHK
  → ACT → ACT_WAIT (wait tRCD cycles after ACTIVATE)
      ├── READ → READ_WAIT (wait tCL cycles)
      |         → PRECHARGE → PRECHARGE_WAIT (wait tRP)
      |         → READ_TO_READ or READ_TO_WRITE (enforce tCCD/tRTW)
      |
      └── WRITE → WRITE_WAIT (wait tCWL cycles)
                → PRECHARGE → PRECHARGE_WAIT (wait tRP)
                → WRITE_TO_WRITE or WRITE_TO_READ (enforce tCCD/tWTR)

  → REF → REF_WAIT (wait tRFC cycles — for REFRESH command)
  → DONE
```

DDR3 Commands issued (RAS_n, CAS_n, WE_n encoding):

- NOP: 1,1,1
- ACTIVATE: 0,1,1 + bank + rowaddress on A
- READ: 1,0,1 + bank + column address on A, A10=0 (no auto-precharge)
- WRITE: 1,0,0 + bank + column address on A, A10=0
- PRECHARGE: 0,1,0 + bank, A10=0 (single bank)
- REFRESH: 0,0,1
- MRS: 0,0,0 + BA[2:0] selects register + A[15:0] = data

Data Burst State Machine (State2 — runs in parallel):

```
IDLE
  → RPREAMBLE (DQS low — 1 cycle preamble before read data)
  → RBURST    (4 DQS cycles = 8 data beats = 64 bytes captured)
  → RPOSTAMBLE (DQS low — 1 cycle postamble)

  → WAIT_1 → WAIT_2 → WAIT_3 (write latency pipeline)
  → WPREAMBLE (DQS low — drive preamble)
  → WBURST    (4 DQS cycles = 64 bytes driven out)
  → WPOSTAMBLE (DQS low — postamble)
```

Bank State Tracking: The module tracks which banks are open (activated) using `bank0_st..bank7_st` registers. This enables future optimization to skip ACT/PRECHARGE for the same open row.

4.6 `ddr3mc_processing_engine.v` — Command Processor

Role: Bridges between flow_control (which delivers decoded rank/bank/row/col) and rw_control (which generates the DDR3 signals). Also manages data movement between WDQ/RDQ and the PHY.

Key functions:

- Instantiates two `ddr3mc_rw_control` modules – one per rank (rank0 and rank1)
- Selects which rw_control is active based on `cmd_rank`
- Pulls write data from WDQ and feeds it to the PHY's `dq_i1/dq_i2` ports
- Captures returned read data from PHY's `dq_q1/dq_q2` ports and pushes to RDQ
- Generates `write_en`, `burst_en`, `preamble_en`, `postamble_en` timing signals
- Monitors `refi_cnt_ok0/1` to trigger refresh when tREFI expires

Data path for WRITE:

```
WDQ → wdq_rd_data[511:0] → split into 4×128-bit chunks →
i1_dq[63:0] + i2_dq[63:0] (two DDR phases per cycle) → PHY ODELAY → DQ pins
```

Data path for READ:

```
DQ pins → PHY IDELAY → IDDRE1 → q1_dq[63:0] + q2_dq[63:0] →
processing_engine assembles 4 cycles × 128 bits = 512 bits → rdq_wr_data → RDQ
```

4.7 `ddr3mc_imux.v` — Input Multiplexer (Combinational)

Role: Pure combinational logic – zero latency, zero clock cycles. Routes DDR3 command signals from the correct source engine to the PHY.

Selection logic:

```verilog
case({init_en, cal_en, proc_en})
  3'b100: // INIT phase — route init_engine outputs
    {ras_n, cas_n, we_n, cs_n, cke, odt, a, ba} = {*_init};
  3'b010: // CAL phase — route cal_engine outputs
     {ras_n, cas_n, we_n, cs_n, cke, odt, a, ba} = {*_cal};
   3'b001: // PROC phase — route processing_engine outputs
    {ras_n, cas_n, we_n, cs_n, cke, odt, a, ba} = {*_proc};
  default: all outputs = safe idle values (CS_n=1, RAS/CAS/WE=1)
endcase
// RESET_n special case: only init_engine controls it
reset_n_imux = init_en ? reset_n_init : 1'b1;
```

This clean separation means each engine is completely independent – no engine needs to knowabout the others.

4.8 `ddr3mc_queues.v` — Three-Queue System

Role: Instantiates three FIFOs that decouple the system bus from the DDR3 timing engine.

Command Queue (CMDQ):

- Width: `CMDQ_WIDTH = SYS_ADDR_WIDTH + SYS_CMD_WIDTH = 33+4 = 37 bits`
- Depth: 4 entries
- Type: `ddr3mc_fifo_sync` — standard synchronous FIFO
- Write: when system posts a valid READ or WRITE command
- Read: by flow_control in CMD_PULL state

Write Data Queue (WDQ):

- Write width: `WDQ_WRITE_DATA_WIDTH = 512 bits` (full system data)
- Read width: `WDQ_READ_DATA_WIDTH = 512/4 = 128 bits` (one DDR beat)
- Depth: 4 entries
- Type: `ddr3mc_fifo_sync_piso` — Parallel-In Serial-Out (512→128 bit conversion)

- Write: when system posts a WRITE command
- Read: by processing_engine 4 times per write (to get all 4 DDR beats)

Read Data Queue (RDQ):

- Write width: `RDQ_WRITE_DATA_WIDTH = 128 bits` (one DDR beat from PHY)
- Read width: `RDQ_READ_DATA_WIDTH = 512 bits` (full 64-byte response)
- Depth: 4 entries
- Type: `ddr3mc_fifo_sync_sipo` — Serial-In Parallel-Out (128→512 bit assembly)
- Write: by processing_engine 4 times as DDR data returns
- Read: by system when `sys_read_data_pull` asserted; `sys_read_data_valid = !rdq_empty`

4.9 `ddr3mc_fifo_sync.v` — Standard Synchronous FIFO

Role: Parameterized FIFO with configurable width and depth. Used for CMDQ.

Features:

- Parameterized: `FIFO_WIDTH` and `FIFO_DEPTH`
- `room_avail` and `data_avail` counters for flowcontrol
- Write pointer / read pointer using `$clog2(FIFO_DEPTH)` wide counters
- Standard `full` and `empty` flags
- Uses `ddr3mc_sram` as the storage element

`ddr3mc_fifo_sync_piso.v` — Parallel-In Serial-Out FIFO

Accepts wide parallel writes, delivers narrowserial reads. Internally stores at write width, then serializes on read side using a counter to select sub-words.

`ddr3mc_fifo_sync_sipo.v` — Serial-In Parallel-Out FIFO

Accepts narrowserial writes, accumulates into wide parallel reads. Collects N narrow writes before marking one wide entry as available.

`ddr3mc_sram.v` — Internal SRAM Storage

Simple dual-port SRAM model: independent write clock and read clock. Synchronous write, asynchronous (combinational) read. Used as storage array inside all FIFOs.

`ddr3mc_fif_sync.v`

A second FIFO file – minor variant of `ddr3mc_fifo_sync.v`, likely used in specific

calibration or debug contexts.

---

5. CALIBRATION SUBSYSTEM — The Most Complex Part

Engineer: Venkateshwara Prashanth Chandrasekaran
Purpose: At DDR3 speeds (200 MHz in this design), sub-nanosecond timing errors corrupt data. All 6 calibration steps must complete before the memory is usable.

5.1 `ddr3mc_cal_engine.v` — Calibration Master (~87KB, largest file)

Role: Top-level calibration instantiator. Instantiates all 6 calibration sub-modules and connects them to the cal_flow_control sequencer.

Contains `ILA_*` debug hooks (commented out by default) for each calibration step that can be enabled with `CALIB_DEBUG` define.

5.2 `ddr3mc_cal_flow_control.v` — Calibration Sequencer

State Machine — executes steps in strict order:

```
IDLE → (cal_en asserted)
  ↓
PREAMBLE_DETECTION → (preamb_done)
  ↓
CLOCK_CENTERING → (ccentrng_done)
  ↓
DQ_DETECTION → (dqdetect_done)
  ↓
WRITE_LEVELING → (wlev_done)
  ↓
WRITE_DQS_CENTERING → (wdqscentrng_done)
  ↓
READ_WRITE_SANITY_CHECK → (rwscheck_done)
  ↓
CAL_DONE (signals ddr3mc_cal_done to flow_control)
```

5.3 `ddr3mc_cal_preamble_det.v` + `_wrapper.v` — Step 1: Preamble Detection

What it does: Finds when DQS transitions from logic-0 to toggling – the "preamble" that signals the start ofvalid DDR3 read data.

Algorithm:

1. Issues PRECHARGE ALL to put all banks in idle state

2. Issues ACTIVATE to open a test row
3. Issues READ command
4. Sweeps DQS IDELAY from tap 0 to max
5. At each tap position, samples `dqs_q1` and `dqs_q2`
6. Finds first tap where DQS is consistently low(preamble region detected)
7. Sets the DQS idelay to this position
8. The `_wrapper` instantiates one detector per DQS lane (8 lanes for 64-bit bus)

Why needed: Without detecting the preamble, the controller cannot distinguish valid data from bus noise, leading to corrupt reads.

5.4 `ddr3mc_cal_clock_centering.v` + `_wrapper.v` — Step 2: Read DQS Centering

What it does: Centers the read DQS strobe within the data eye for reliable read data capture.

Algorithm:

1. Writes a known pattern to DDR3
2. Sweeps DQS IDELAY through all tap positions
3. At each tap, reads back the pattern and checks ifit matches
4. Finds left edge (where data first becomes correct) and right edge (where it last correct)
5. Sets DQS idelay to `(left_edge + right_edge) / 2` — the center ofthe eye
6. The `_wrapper` handles all 8 DQS lanes independently (each lane may have different optimal position)

5.5 `ddr3mc_cal_dq_detect.v` + `_wrapper.v` — Step 3: Per-Bit DQ Detection

What it does: Aligns each individual DQ bit within each byte lane.

Algorithm:

1. Writes a walking-ones pattern and other test patterns
2. For each of64 DQ bits independently, sweeps that bit's IDELAY
3. Finds where that specific bit is reliably captured
4. Sets per-bit delays to align all 64 bits to the DQS strobe

Why needed: Even within one byte lane, different DQ bits may have slightly different PCB trace lengths causing skew. Per-bit calibration eliminates this to the resolution ofone delay tap.

5.6 `ddr3mc_cal_write_leveling.v` + `_wrapper.v` — Step 4: Write Leveling

What it does: The most fundamental calibration. DDR3 uses fly-by clock topology where the system clock arrives at each DRAM chip at a different time due to trace length differences.

Algorithm:

1. Sets DDR3 into write leveling mode via MR1 bit A7=1
2. In write leveling mode, each DDR3 chip samples DQS against its local CK and returns 1 if DQS arrived after CK rising edge, 0 ifbefore
3. Sweeps DQS ODELAY (output delay) for each byte lane
4. Finds the delay value where feedback transitions from 0 to 1 — this is exact CK-to-DQS alignment
5. Exits write leveling mode by clearing MR1 A7

Input ports: `dqs_odelay_cntvalueout` (current delay readback), `dq_q1/dq_q2` (DQ data capturing the DRAM's feedback)

Why needed: Without write leveling, all write operations fail because DQS signals are not synchronized to the DRAM's local clock.

5.7 `ddr3mc_cal_write_dqs_centering.v` + `_wrapper.v` — Step 5: Write DQS Centering

What it does: After write leveling finds rough DQS-to-CK alignment, this step precisely centers write DQS within the write data eye.

Algorithm:

1. Writes test patterns while sweeping DQS ODELAY
2. Reads back each pattern and checks correctness
3. Finds left and right margins ofthe write data eye
4. Sets DQS output delay to center position

5.8 `ddr3mc_cal_read_write_sanity_check.v` + `_wrapper.v` — Step 6: Final Verification

What it does: Runs comprehensive read/write verification across multiple addresses, banks, and ranks to confirm all 6 calibration steps succeeded.

Test patterns used:

- All zeros: 0x000...000
- All ones: 0xFFF...FFF
- Alternating: 0xAAA...AAA, 0x555...555
- Walking ones and walking zeros patterns

Reports `rwscheck_done` only when all patterns pass. Outputs `err` flag if any mismatch detected.

5.9 `ddr3mc_cal_imux_pipe.v` — Calibration Input Mux Pipeline

Routes captured DQS/DQ data from PHY to the currently active calibration step. Uses the `state` register from cal_flow_control to select which step's data input is active. Adds pipeline registers for timing closure.

5.10 `ddr3mc_cal_omux_pipe.v` — Calibration Output Mux Pipeline

Routes DDR3 command outputs from the active calibration step to the imux (and then to the PHY). Adds pipeline registers for timing closure.

5.11 `ddr3mc_iodelay_control.v` — Delay Element Controller

Translates the calibration engine's delay value requests (absolute tap count values) into the correct `ce`, `inc`, `load`, `cntvaluein` control sequences required by the IDELAY/ODELAY elements in the custom PHY. Each delay element requires a specific control protocol to update its tap value.

---

6. PHYSICAL LAYER (PHY) — Custom ASIC Silicon

This is what makes this project unique vs an FPGA implementation. All IO cells are custom GUC (Global Unichip Corporation) standard cells designed for a TSMC process node.

6.1 `ddr3mc_phy_top.v` — PHY Top Level

Role: Aggregates all PHY sub-components. Presents a clean digital interface to the controller side, and connects to actual DDR3 silicon pins on the other side.

Instantiates:

- 8× `ddr3mc_phy_byte_lane` — one per data byte (64-bit bus = 8 bytes)
- 1× `ddr3mc_phy_control_lanes` — all command/address/control signals
- 2× `ddr3mc_phy_ck_generator` — CK0 (rank0) and CK1 (rank1) clocks
- 1× `ddr3mc_phy_pvt_cal` — PVT impedance calibration

Key signal: `phy_ready` — asserted only when PVT calibration passes. `ddr3mc_flow_control` waits for this before starting CFG_EN.

6.2 `ddr3mc_phy_byte_lane.v` — Complete Data Byte Lane

Role: Handles one complete byte of the DDR3 interface: 8 DQ bits + 1 DQS pair + 1 DM bit.

Power supply ports (ASIC-specific): `ddr_vddp`, `ddr_vdd`, `ddr_vssp`, `ddr_vss` — explicit power rail connections required in ASIC design.

Global control ports: `pwr_ok_hv`, `drv_p/n_hv` (output drive strength), `odt_p/n_hv` (termination), `pdio_bus[3:0]`, `puio_bus[3:0]` (impedance codes from PVT cal), `ddr4` (DDR4 vs DDR3 mode), `vref_int`, `esd_dr`

Instantiates per byte lane:

- 4× `ddr3mc_phy_dq_bit_lane` — handles 2 DQ bits each (4×2 = 8 DQ bits per byte)
- 1× `ddr3mc_phy_dqs_bit_lane` — DQS strobe with IDELAY+ODELAY+DDR IO
- 1× `ddr3mc_phy_dm_bit_lane` — Data Mask bit
- Each sub-lane has: `DLL_TOP_S4` (DQ) or `DLL_TOP_S1` (DQS) – custom DLL for delay calibration

6.3 `ddr3mc_phy_dq_bit_lane.v` — Individual DQ Bit

Role: One DQ bit with complete bidirectional DDR IO.

Key operations:

- **Write path:** `i1[bit]` (rising edge data) + `i2[bit]` (falling edge data) → `ODDRE1` DDR output register → ODELAY (fine output timing) → IOBUF (bidirectional pad)
- **Read path:** IOBUF → IDELAY (fine input timing adjusted by calibration) → `IDDRE1` DDR input register → `q1[bit]` (rising edge capture) + `q2[bit]` (falling edge capture)
- `dq_output_enable` controls IOBUF direction (1=input/read, 0=output/write – active lowin ASIC convention)

6.4 `ddr3mc_phy_dqs_bit_lane.v` — DQS Strobe Bit

Role: The DQS strobe with special inverted input to support write leveling.

Critical note from code: The DQS bit lane has a separate `dqs_in` input (the PRBS/calibration pattern) used specifically during write leveling to receive DRAM feedback without interfering with normal DQS operation.

Write mode: DQS driven differentially with preamble (low) → toggle (clock-like) → postamble (low)
Read mode: DQS received differentially, IDELAY centered in data eye by clock centering calibration

6.5 `ddr3mc_phy_dm_bit_lane.v` — Data Mask Bit

Role: Output-only DM signal for selective byte write masking. Has ODELAY for alignment to write DQS. In this implementation `dm_i1 = dm_i2 = 0` (all bytes always written –

masking not actively used).

6.6 `ddr3mc_phy_ck_generator.v` — DDR3 Clock Generator

Role: Generates the differential DDR3 clock pairs (CK0_p/n for rank0, CK1_p/n for rank1).

```
i1 = 0, i2 = 1 (permanently)
→ ODDRE1: outputs 0 on rising edge, 1 on falling edge = differential clock
→ ODELAY: fine clock timing adjustment
→ Differential output pad: CK_p = clock, CK_n = inverted clock
```

The `CKINVERT` parameter mentioned in README ("ckinv250") inverts the clock output for specific board/signal integrity requirements.

6.7 `ddr3mc_phy_control_lanes.v` — Command/Address/Control Lane

Role: Drives all DDR3 control signals through the ASIC IO cells. Uses parameterized widths for `DDR3_A_WIDTH=16`, `DDR3_BA_WIDTH=3`, `CFG_NUM_DIMMS=1`.

Signals driven: A[15:0], BA[2:0], RAS_n, CAS_n, WE_n, RESET_n, CKE0, CKE1, ODT0, ODT1, CS0_n, CS1_n

All control signals are registered and driven through `OBUF` output cells from the `IGIDDRT01A` library.

6.8 `ddr3mc_phy_pvt_cal.v` — PVT Impedance Calibration

Role: Calibrates IO cell output driver impedance against Process, Voltage, and Temperature variations using an external ZQ precision resistor (typically 240Ω).

Instantiates:

- The ZQ IO cell from IGIDDRT01A (special cell connected to external resistor)
- `cal.v` — the main PVT calibration algorithm

Output signals distributed to all IO cells:

- `puio[3:0]` — pull-up impedance control code
- `pdio[3:0]` — pull-down impedance control code
- `pvt_cal_err` – asserted if ZQ resistor not detected or calibration fails

---

7. CUSTOMASIC CELL LIBRARY (GUC)

These files model actual silicon transistor circuits in behavioral Verilog for simulation.

The complete custom DDR IO cell library. Contains:

- **Power pad cells:** `PVDDP_H`, `PVSSI_H`, `PVDDPI_H`
- **Bidirectional DQ cells:** `IOBUF_*` – for DQ signals (read during read, write during write)
- **Output-only cells:** `OBUF_*` – for CK, DM, address, command signals
- **Input-only cells:** `IBUF_*` — for EVENT_n (DIMM alert input)
- Multiple drive strength variants for each cell type

All cells have:

- ESD protection circuits (modeled behaviorally)
- Programmable pull-up/pull-down via `puio_bus/pdio_bus[3:0]`
- Programmable drive strength via `drv_p/n_hv[2:0]`
- On-Die Termination control via `odt_p/n_hv[2:0]`
- Power-OK monitoring via `pwr_ok_hv`

7.2 `DLL_DCC_v003.v` — Delay Locked Loop with Duty Cycle Correction

Role: The analog timing heart ofthe PHY. Locks to the input clock period and generates calibrated delay values.

Components:

- `IGADLLT04A_MASTER_v003` – measures clock period, outputs LEAD/LAG
- Up to 5× `IGADLLT04A_SLAVE_v003` – each produces different phase fractions
- DCC (Duty Cycle Correction) – ensures exactly 50% duty cycle

`dll_lock` **output** – asserted when DLL has converged. `phy_ready` depends on this.

7.3 `IGADLLT04A_MASTER_v003.v` — DLL Master Cell

Measures the reference clock period using a delay chain. Outputs `LAG` (delay too long) and `LEAD` (delay too short) to a control loop that adjusts `ADJM[8:0]` until locked.

7.4 `IGADLLT04A_SLAVE_v003.v` — DLL Slave Cell (~50KB, second largest file)

The actual analog delay line each data signal passes through. `ADJ[8:0]` gives 512 tap positions. With the design running at 200 MHz (5000ps period), this gives approximately 5000/512 ≈ 10ps per tap resolution. The README mentions "250 taps" referring to half-period (2500ps) delay range.

- **Match:** Ensures equal routing delay between master and slave paths
- **Phase Detector:** Compares two signal phases – core ofthe DLL feedback loop

7.6 `DLL_TOP_S1.v` and `DLL_TOP_S4.v` — DLLTop Wrappers

- `DLL_TOP_S1` — Single-bit DLL for DQS and CK lanes
- `DLL_TOP_S4` — 4-bit parallel DLL for DQ groups (4 DQ bits share one DLL)

7.7 `DELAY_S1.v` and `DELAY_S4.v` — Delay Cell Models

Single-bit and 4-bit behavioral models ofthe analog delay elements used within IO cells.

7.8 `iddre1.v` — Input DDR Register

```verilog
// Captures on BOTH clock edges — DDR (Double Data Rate)
always @(posedge C) q1 <= d;  // rising edge capture
always @(negedge C) q2 <= d;  // falling edge capture
```

Combined, q1 and q2 give two data samples per clock period, achieving the 2× data rate.

7.9 `oddre1.v` — Output DDR Register

```verilog
// Drives on BOTH clock edges
always @(posedge C) q_rise <= d1;   // drives d1 on rising edge
always @(negedge C) q_fall <= d2;   // drives d2 on falling edge
```

7.10 `cal.v` — ZQ Calibration Algorithm

The digital state machine that controls the PVT calibration loop:

- `pcal_en` enables pull-up calibration (P-channel), monitors `pcomp` feedback
- `ncal_en` enables pull-down calibration (N-channel), monitors `ncomp` feedback
- State machine sweeps `pu[3:0]` and `pd[3:0]` codes until comparators showmatch
- `pu_giveup_flag` and `pd_giveup_flag` indicate calibration failure
- `upd_cal_req/ack` protocol allows periodic ZQ recalibration during normal operation (every ~300ms per JEDEC spec)

Calibration parameters:

- `T_PWR_ON = 9'd5` — wait cycles after power on
- `TS_CODE_P_A/B/C/D` — phase step sizes for pull-up search (2, 100, 200, 300 taps)
- `TH_CODE_P = 9'd2` — hysteresis to prevent oscillation

---

8. MEMCTRL — SYSTEM WRAPPER

8.1 `memory_controller_top.sv` — System Integration Top (SystemVerilog)

Role: The chip-level top that connects to the RISC-V (or other CPU) system bus, instantiates DDR3MC, SFC, and SPDC, and connects external IOs.

Additional interfaces vs DDR3MC:

- Differential input clock: `gclk_p/gclk_n` (board clock input)
- I2C: `i2c_scl/sda/sa` (shared between SPDC and temperature sensor)
- SPI: 4 sets of SPI pins (spi_sck/cs/miso/mosi/hold_n/wp_n A/B/C/D) for 4 flash chips
- `memc_sfc_rp_en/sa/ea/done` — boot flash reprogramming interface
- `spd_reg0/1/2` — 2048-bit SPD register outputs from SPDC
- Optional TMB (Test Memory Bus) interface via `USE_TMB` define

CPU System Bus (decoupled ready/valid):

```
CPU → mem_req_valid, mem_req_ready, mem_req_addr[32: 0],
      mem_req_cmd[3: 0], mem_req_data[511: 0], mem_req_mask[63: 0]
CPU ← mem_resp_valid, mem_resp_ready, mem_resp_data[511: 0]
```

8.2 `memory_controller_interface_to_ddr3mc.v` — Protocol Bridge

Role: Converts between the decoupled system bus protocol and the DDR3MC's direct command interface.

Two internal state machines:

Request FSM:

```
REQ_IDLE → REQ_SCRAMBLE (scramble the write data with LFSR)
         → REQ_CMD (post command + address to DDR3MC)
         → REQ_WAIT (wait for DDR3MC ready)
```

Response FSM:

```
RESP_IDLE → RESP_SCRAMBLE (descramble the read data)
          → RESP_WAIT_READY (wait for consumer ready)
          → RESP_WAIT (complete)
```

Instantiates `scrambler` for both request (write) data encryption and response (read) data decryption. The same 3-bit LFSR polynomial ensures symmetric scramble/descramble.

8.3 `RRArbiter.v` — Round-Robin Arbiter (Chisel-generated)

Role: Arbitrates between two CPU request ports fairly.

Algorithm: Uses a 1-bit register `_T_1` that remembers the last chosen port. Alternates between port 0 and port 1 when both have pending requests. Pure combinational grant logic — zero latency from request to grant.

Ports: `io_in_0` (port 0 request), `io_in_1` (port 1 request), `io_out` (arbitrated output), `io_chosen` (0=port0 won, 1=port1 won)

This is auto-generated from a Chisel hardware description (Scala-style annotations visible in comments: `@[Arbiter.scala 107:25]`).

8.4 `Queue.v` (QueueCompatibility) — Response Ordering Queue

Role: 2-entry FIFO that tracks which port each outstanding request belongs to, ensuring in-order responses.

Also Chisel-generated. Maintains a circular buffer ofport IDs, written when a request is issued and read when the response arrives, ensuring each response goes back to the correct requester.

8.5 `memory_controller_fpga_test_driver.sv` — Automated FPGA Test

Generates systematic read/write test patterns when synthesized on FPGA for hardware validation. Not used in ASIC.

8.6 `memory_controller_fpga_wrap.sv` — FPGA Wrapper

Wraps the complete MemCtrl+DDR3MC for FPGA validation boards. Adds Xilinx MMCM for clock generation, FPGA-compatible IO, and board-specific connectivity.

8.7 `memory_controller_tmb.sv` — Test Memory Bus

Interfaces to DINI FPGA PCIe board for board-level testing. Allows a laptop to send memory transactions via PCIe to the FPGA running the controller, enabling hardware-in-loop testing.

Purpose: Manages a SPI NOR Flash memory chip for boot code storage. On startup, copies firmware from Flash into DDR3 RAM.

File Contributions:

File	Role
`sfc_top.v`	Top – connects SPI flash to DDR3MC
`sfc_spim.v`	SPI Master orchestrator – selects read/write/erase
`sfc_spim_ctrl.v`	Low-level SPI bit-bang controller
`sfc_spim_read_control.v`	Issues READ (0x03) or Fast Read (0x0B) with address
`sfc_spim_write_control.v`	Issues WRITE ENABLE then PAGE PROGRAM (0x02)
`sfc_spim_erase_control.v`	Issues SECTOR ERASE (0xD8) or BLOCK ERASE
`sfc_spim_top.v`	Connects ctrl + read/write/erase + data FIFOs
`sfc_cmd_generator.v`	Converts high-level ops to SPI command sequences
`sfc_ddr3mc_transporter.v`	Moves data between SPI FIFO and DDR3 memory
`sfc_io.v`	IO pad interface for SPI SCK/MOSI/MISO/CS/WP/HOLD
`ddr3mc_dummy.sv`	Stub for DDR3MC in standalone SFC testbench
`reset_sync.v`	Two-flop reset synchronizer for SFC clock domain

SFC Boot Sequence:

```
Assert rp_en with rp_sa (start addr) and rp_ea (end addr)
    → sfc_cmd_generator generates READ commands from flash
  → sfc_spim sends SPI READ: [CMD=0x03][A23: A16][A15: A8][A7: A0][DATA. . . ]
  → sfc_ddr3mc_transporter writes data to DDR3 at address starting 0x0
  → After rp_ea reached: assert rp_done
  → memory_controller_top asserts memCtrl_rdy only after rp_done
```

Supports both: Micron MT25QU01G and Macronix MX66U1G45G 1Gb SPI NOR flash chips (vendor models included in sim/).

FIFO Bridge: Three async FIFOs bridge between SPI clock domain (~50MHz) and DDR3 clock domain (200MHz): `fifo_async`, `fifo_async_piso`, `fifo_async_sipo` with Gray-code pointer synchronization.

---

10. SPDC — SPD DECODER CONTROLLER

Purpose: Reads the 256-byte EEPROM on DDR3 DIMM modules via I2C and decodes the timing parameters and geometry automatically configuring the memory controller.

File Contributions:

File	Role
`spdc_top.v`	Top FSM – manages I2C read sequence
`spdc_i2c_clk.v`	Generates I2C SCL clock (typically 100KHz or 400KHz)
`spdc_i2c_st.v`	I2C state machine: START, address, data, ACK, STOP
`spdc_i2c_top.v`	Combines clock + state machine into complete I2C master
`spdc_dec.v`	Decodes raw SPD bytes into timing parameters in clock cycles
`spdc_dec_beta.v`	Beta/improved version ofdecoder
`spdc_pipeline.v`	Pipeline registers for decoded output
`spdc_ctrl.v`	Controls overall SPD reading flow and bank switching
`spdc_qadd.v`	Q-format fixed-point adder
`spdc_qdiv.v`	Q-format fixed-point divider
`spdc_qmults.v`	Q-format fixed-point multiplier
`spdc_testDrv.sv`	Test driver for standalone verification
`spdc_wrap.sv`	FPGA wrapper for board validation

SPDC Operation:

1. Read SPD EEPROM at I2C address 0x50 (base address, SA pins set address offsets)
2. spdc_top FSM: IDLE → SPD_WRITE_CMD → WAIT → SPD_READ_CMD → SPD_READ_DATA_VALID → PARSE → DEC_READ_CMD
3. Each byte read goes into reg0/reg1/reg2 (three DIMMs, 2048 bits each = 256 bytes each)
4. spdc_dec decodes bytes:
   - Byte 12: tCK_avg_min (minimum clock cycle time in MTB units)
   - Byte 13: tAA_min (min CAS-to-data time)
   - Byte 14: tWR_min (min write recovery time)
   - Byte 15: tRCD_min (min RAS-to-CAS delay)
   - Byte 20: tRP_min (min row precharge time)
   - Bytes 24, 23: tRC_min
   - Bytes 26, 25: tRFC_min (min refresh recovery time)
   - Bytes 29, 28: tFAW_min
5. Fixed-point arithmetic: converts MTB units (0.125ns) to clock cycles
   - spdc_qdiv divides timing values by clock period to get nCK values
6. Outputs to memory_controller_top as cfg registers

Note in code: *"This design works for all memory which has FTB byte 9 value less than 1 and bytes 34-38 as zero. Does not work if FTB is greater than 0.001 value."* — documenting a known limitation.

After SPD reading completes, the same I2C bus (`TS_SDA`/`TS_SCL`) is reused for the DIMM temperature sensor monitoring.

---

11. SUPPORT MODULES

`scrambler.v` — 512-bit LFSR Data Scrambler

Algorithm: 3-bit Linear Feedback Shift Register with polynomial x^3+x+1

```verilog
// Each group of 4 bits in the 512-bit word is XOR'd with LFSR pattern:
data_c[0] = data_in[0] ^ lfsr_q[2];
data_c[1] = data_in[1] ^ lfsr_q[1] ^ lfsr_q[2];
data_c[2] = data_in[2] ^ lfsr_q[0] ^ lfsr_q[1];
data_c[3] = data_in[3] ^ lfsr_q[0];
// Pattern repeats for all 512 bits...
```

Two purposes:

1. **EMI reduction** – prevents repetitive data patterns that cause concentrated RF emissions at specific frequencies
2. **Reliability** – prevents all-zeros/all-ones patterns that stress DRAM cells

`scram_rst` resets LFSR to all-ones initial state. `scram_en` enables scrambling. The scrambler is symmetric – same operation both scrambles and descrambles (XOR is self-inverse when LFSR is reset to same initial state).

`pipeline.v` — Pipeline Register

Simple configurable-depth pipeline register. Used to add latency stages for timing closure between modules operating at different pipeline depths.

`reset_sync.v` — Reset Synchronizer

Two-flop synchronizer to safely propagate the asynchronous external reset into each clock domain. Prevents metastability when reset is released asynchronously relative to the clock.

---

12. SIMULATION ENVIRONMENT

`ddr3.sv` — DDR3 DRAM Behavioral Model (~165KB, largest file)

Complete SystemVerilog behavioral model ofa DDR3 SDRAM chip. **This is not synthesizable** – it's a simulation-only model that mimics exactly howa real DDR3 chip behaves.

What it models:

- All DDR3 commands: ACTIVATE, READ, WRITE, PRECHARGE, REFRESH, MRS, ZQ, POWER DOWN
- Full timing violation checking (prints errors iftRCD, tRP, tWTR etc. violated)
- Write leveling mode (DRAM responds to DQS with DQ feedback)
- Full 2D memory array storage and retrieval during simulation
- ODT (On-Die Termination) assertion/deassertion timing
- Temperature-dependent timing (from `4096Mb_ddr3_parameters.svh` or `8192Mb_ddr3_parameters.svh`)

`ddr3_sim_sodimm_x8_2R.sv` — SO-DIMMModel

Models a complete 2-rank SO-DIMM by instantiating multiple `ddr3.sv` instances together, correctly wiring the shared bus.

Behavioral model ofthe DIMM's SPD EEPROM. Reads initial contents from `eeprom.txt` and serves them via simulated I2C when addressed. `eeprom.txt` contains 256 hex bytes representing actual DDR3-1600 DIMM SPD data.

`24AA02.v` — Microchip EEPROM Model

Vendor-provided behavioral model of Microchip 24AA02 2Kbit I2C EEPROM – the specific chip used as SPD EEPROM.

`ddr3mc_tb_top.sv` and `tb_ddr3mc_top.sv` — Main Testbenches

Top-level simulation testbenches:

- Instantiate `ddr3mc_fpga_wrap` (DUT) + DDR3 DIMM model + SPD EEPROM model
- Generate 200 MHz system clock (5000ps period)
- Apply reset sequence
- Write configuration registers
- Wait for `sys_status` to show INIT_DONE, CAL_DONE, RW_READY
- Post test read/write transactions
- Verify read data matches written data
- Report PASS/FAIL

`ddr3mc_sim_bfm.sv` — Bus Functional Model

SystemVerilog BFM with tasks:

- `mem_write(addr, data)` – drives system bus write transaction
- `mem_read(addr)` – drives system bus read transaction, returns data
- `cfg_write(addr, data)` – writes configuration register

`tb_phy_top.sv` — PHYIsolation Testbench

Testbench that exercises only `ddr3mc_phy_top` in isolation, applying direct DQ/DQS stimulus to verify delay element behavior and DDR capture without running the full controller.

`4096Mb_ddr3_parameters.svh` and `8192Mb_ddr3_parameters.svh`

JEDEC-standard timing parameter definitions for 4Gb and 8Gb DDR3 densities. Included by `ddr3.sv` to configure the simulation model for the correct DRAM density.

TCL Scripts (`DDR3MC/scripts/`)

Script	Purpose
`ddr3mc_envsetup.tcl`	Defines compilation file list and order for simulation
`ddr3mc_define.tcl`	Project-level variable definitions, top module names
`ddr3mc_ip.tcl`	Packages DDR3MC as Vivado IP core for block design use

XDC Constraint Files (`DDR3MC/const/`)

Named by board revision and channel count:

- `ddr3mc_io_F4A_fpgaBD_x1.xdc` — F4A board, fpgaBD variant, 1 DDR3 channel
- `ddr3mc_io_F4A_fpgaBD_x3.xdc` — F4A board, fpgaBD variant, 3 DDR3 channels
- `ddr3mc_io_F1A_fpgaA_x3.xdc` — F1A board, fpgaA variant, 3 channels
- `ddr3mc_wrap_*.xdc` — Wrapper-level constraints for full system

Contain: FPGA pin assignments, I/O standards (SSTL15 for DDR3), drive strengths, slew rates, input/output delay constraints for timing analysis.

`header.sh` + **`header.txt`**

Shell script that auto-adds the company copyright header to new RTL files.

---

14. COMPLETE FUNCTION FLOW

Power-Up to Ready State Machine

```
POWER APPLIED
      |
      ▼
[sys_clk_locked = 0]
PHY DLL locks → phy_ready asserted
      |
      ▼ (sys_clk_locked && phy_ready)
═══════════════════════════════════
PHASE 1: CONFIGURATION (CFG_EN)
═══════════════════════════════════
```

ddr3mc_cfg accepts writes to:
  - Load timing parameters (tCL=6, tCWL=5, tRCD=3 etc.)
  - Load DIMM geometry (16-bit row, 11-bit col, 3-bit bank)
  - Set address ranges for rank0 and rank1
  - Set mode register values (DIC, RTT etc.)
→ cfg_done asserted
      |
      ▼
═══════════════════════════════════
PHASE 2: DDR3 INITIALIZATION (INIT_DIMM0_EN)
═══════════════════════════════════
ddr3mc_init_engine executes:
  RESET_n = 0 → wait 200µs
  RESET_n = 1 → wait 500µs
  CKE = 1 → NOP → wait tXPR
  Write MR2 (CWL=5) → wait tMRD
  Write MR3 (MPR off) → wait tMRD
  Write MR1 (DIC=RZQ/7, RTT=RZQ/6) → wait tMRD
  Write MR0 (CL=6, DLL reset) → wait tMOD
  ZQCL → wait 512 cycles
 [repeat for rank1]
→ init_done asserted
      |
      ▼
═══════════════════════════════════
PHASE 3: PHY CALIBRATION (CAL_EN)
═══════════════════════════════════
ddr3mc_cal_engine executes sequentially:
  Step 1: Preamble Detection
    → Find DQS preamble for all 8 byte lanes
    → Set dqs_idelay per lane
  Step 2: Clock Centering
    → Center DQS idelay in read data eye
    → Per byte lane, find left+right eye margins, set to center
  Step 3: DQ Detection
    → Per-bit DQ idelay alignment
    → 64 individual DQ bits aligned to DQS strobe
  Step 4: Write Leveling
    → Measure CK-to-DQS alignment via fly-by compensation
    → Sweep dqs_odelay until DRAM feedback shows alignment
  Step 5: Write DQS Centering
    → Center write DQS in write data eye
  Step 6: Read/Write Sanity Check
    → Write/read all-0, all-1, alternating patterns

```
      → Verify all 512 bits correct
→ cal_done asserted
        |
        ▼
═══════════════════════════════════
PHASE 4: NORMAL OPERATION
sys_status[RW_READY]  =  1
═══════════════════════════════════
Continuous loop:
  ┌─ Check refresh needed? ──────────────────┐
  |   YES: Issue REFRESH to rank0 or rank1   |
  |        Wait tRFC cycles                  |
  |                                          |
  └─ Command in CMDQ? ──────────────────────►┘
     YES: Pull command (rank/bank/row/col)
          |
          ├── READ:  ACT→READ→PRE (DQ captured by PHY)
          |          Data assembled in RDQ
          |          sys_read_data_valid asserted
          |
          └── WRITE: ACT→WRITE→PRE (DQ driven by PHY)
                     Data consumed from WDQ
```

---

15. COMPLETE DATA FLOW— READ OPERATION

```
CPU
 | mem_req(addr=0x80001000, cmd=READ, valid=1)
 ▼
memory_controller_top.sv (MemCtrl)
 | RRArbiter selects between Port0/Port1
 | QueueCompatibility tracks port ordering
 ▼
memory_controller_interface_to_ddr3mc.  v
 |  REQ_SCRAMBLE: descramble already-scrambled address? No → pass through
 | REQ_CMD: post {CMD_READ, 0x80001000} to CMDQ
 | cmdq_wr_en=1, cmdq_wr_data={4'b1010, 33'h80001000}
 ▼
ddr3mc_queues.v → CMDQ (ddr3mc_fifo_sync, depth=4)
 | Stores {CMD[3:0], ADDR[32:0]} = 37-bit entry
 ▼
ddr3mc_flow_control.v
```

```
  | State: WAIT_CMD_QUEUE_EMPTY → CMD_PULL → CMD_PULL_WAIT
  |  Address decode: 0x80001000 →
  |    rank=0 (falls in dimm0_rank0_sa..ea range)
  |    bank=0 (bits [13:11])
  |    row=0x0800 (bits [10:0+16] → bits [26:11])... etc.
  | cmd_op=READ, cmd_rank=0, cmd_bank=0, cmd_row=X, cmd_col=Y
  | State: RW_EN → proc_en=1
 ▼
ddr3mc_processing_engine.v
  |  Selects rw_control_inst0 (rank0)
  | Passes cmd_bank, cmd_row, cmd_col
 ▼
ddr3mc_rw_control.v (rank0 instance)
  | State1: IDLE → ST_CHK → ACT → ACT_WAIT
  |    Outputs: RAS_n=0, CAS_n=1, WE_n=1, BA=bank, A=row_addr
  |    Wait tRCD=3 cycles
  | State1: ACT_WAIT → READ
  |    Outputs: RAS_n=1, CAS_n=0, WE_n=1, BA=bank, A=col_addr
  | State2: IDLE → RPREAMBLE → RBURST (4 DQS cycles) → RPOSTAMBLE
  |    preamble_en, burst_en, postamble_en timing signals asserted
  | State1: READ_WAIT → PRECHARGE → PRECHARGE_WAIT → DONE
 ▼
ddr3mc_imux.v (proc_en=1, routes proc outputs)
  | ras_n_imux, cas_n_imux, we_n_imux, a_imux, ba_imux ← from proc engine
 ▼
ddr3mc_phy_top.v
  | ddr3mc_phy_control_lanes drives: A, BA, RAS_n, CAS_n, WE_n, CKE, CS_n → DDR3
pins
  | ddr3mc_phy_ck_generator continuously driving CK0_p/n
 ▼
DDR3 DRAM CHIP (Physical Device)
  | After tCL=6 cycles from READ command:
  | DQS[7:0] goes LOW (preamble) for 1 tCK
  | DQS[7:0] toggles 4 times (4 DQS cycles)
  | DQ[63:0] valid on each DQS edge (8 beats × 8 bytes = 64 bytes)
 ▼
ddr3mc_phy_byte_lane.v (×8 instances, one per byte)
  | For each byte lane:
  |    IOBUF receives DQ[7:0] from DRAM
  |    IDELAY adjusts input timing (set by clock centering calibration)
  |    IDDRE1: captures q1[7:0] (rising DQS edge) + q2[7:0] (falling DQS edge)
  |    DQS_bit_lane: captures DQS timing reference
 ▼
ddr3mc_processing_engine.v
```

```
  | Collects 4 cycles of q1_dq[63:0] + q2_dq[63:0]
  | = 4 × 128 bits = 512 bits (64 bytes) total
  | rdq_wr_en=1, rdq_wr_data[127:0] (written 4 times)
  ▼
ddr3mc_queues.v → RDQ (ddr3mc_fifo_sync_sipo)
  | 4 narrow writes (128-bit each) assembled into 1 wide read (512-bit)
  ▼
sys_read_data_valid = 1 (rdq not empty)
  ▼
memory_controller_interface_to_ddr3mc. v
  | RESP_SCRAMBLE: XOR data with LFSR for descramble
  | RESP_WAIT_READY: assert resp_valid
  ▼
memory_controller_top. sv
  | Queue confirms correct port for this response
  | RRArbiter routes response to Port0 or Port1
  ▼
CPU receives mem_resp_data[511:0] ✓
```

---

16. COMPLETE FILE REFERENCE TABLE

DDR3MC/include/

File	Purpose
`ddr3mc_user_data.vh`	Master config: clk period, DIMM count, ranks, electrical params, command opcodes
`ddr3mc_timing_parameters.vh`	JEDEC timing values in clock cycles, DIMM sizes, CFGREG addresses
`ddr3mc_size_parameters.vh`	Bus widths, address widths, queue depths, DDR3 interface widths
`ddr3mc_phy_parameters.vh`	PHY delay tap parameters, DLL config, IO cell parameters

DDR3MC/rtl/ — Controller Logic

File	Purpose
`ddr3mc_top.v`	Absolute top: instantiates all 8 sub-modules, routes all wires

File	Purpose
`ddr3mc_flow_control.v`	Master FSM: sequences CFG→INIT→CAL→NORMAL, address translation, refresh management
`ddr3mc_cfg.v`	Configuration register file: accepts writes for timing and DIMM geometry
`ddr3mc_init_engine.v`	DDR3 power-up initialization: 25-state FSM, programs MR0-MR3, ZQCL
`ddr3mc_processing_engine.v`	Normal R/W: selects rw_control per rank, manages WDQ/RDQ data movement
`ddr3mc_rw_control.v`	DDR3 command sequencer: ACT/READ/WRITE/PRE/REF with exact timing enforcement
`ddr3mc_imux.v`	Command multiplexer: routes init/cal/proc command outputs to PHY
`ddr3mc_queues.v`	Three-queue system: CMDQ (37-bit×4), WDQ (PISO 512→128), RDQ (SIPO 128→512)
`ddr3mc_datapath.v`	Data width conversion and alignment between system bus and PHY
`ddr3mc_fifo_sync.v`	Synchronous FIFO (parameterized width/depth) – used for CMDQ
`ddr3mc_fifo_sync_piso.v`	Parallel-In Serial-Out FIFO – write data width reduction
`ddr3mc_fifo_sync_sipo.v`	Serial-In Parallel-Out FIFO – read data width expansion
`ddr3mc_fif_sync.v`	Alternate FIFO variant used in specific contexts
`ddr3mc_sram.v`	Dual-port synchronous SRAM – storage inside FIFOs

DDR3MC/rtl/ — Calibration

File	Purpose
`ddr3mc_cal_engine.v`	Calibration master: instantiates all 6 cal step
`ddr3mc_cal_flow_control.v`	Cal sequencer FSM: PREAMB→CLKCENTER→DQDET→WLEV→

File	Purpose
`ddr3mc_cal_preamble_det.v`	Step 1: Find DQS preamble, set idelay to pre
`ddr3mc_cal_preamble_det_wrapper.v`	Instantiates preamble detector for all 8 DQS
`ddr3mc_cal_clock_centering.v`	Step 2: Center DQS idelay in data eye
`ddr3mc_cal_clock_centering_wrapper.v`	Multi-lane wrapper for clock centering
`ddr3mc_cal_dq_detect.v`	Step 3: Per-bit DQ idelay alignment
`ddr3mc_cal_dq_detect_wrapper.v`	Multi-bit wrapper for DQ detection
`ddr3mc_cal_write_leveling.v`	Step 4: Write leveling via MR1 feedback, swe
`ddr3mc_cal_write_leveling_wrapper.v`	Multi-lane wrapper for write leveling
`ddr3mc_cal_write_dqs_centering.v`	Step 5: Center write DQS in write data eye
`ddr3mc_cal_write_dqs_centering_wrapper.v`	Multi-lane wrapper for write DQS centering
`ddr3mc_cal_read_write_sanity_check.v`	Step 6: Final R/W verification with multiple t
`ddr3mc_cal_read_write_sanity_check_wrapper.v`	Multi-rank wrapper for sanity check
`ddr3mc_cal_imux_pipe.v`	Routes captured DQ/DQS data to active cali
`ddr3mc_cal_omux_pipe.v`	Routes active cal step DDR3 commands to P
`ddr3mc_iodelay_control.v`	Translates delay tap values into ce/inc/load/

DDR3MC/rtl/ — PHY

File	Purpose
`ddr3mc_phy_top.v`	PHY top: 8×byte_lane + control_lanes + 2×ck_gen + pvt_cal
`ddr3mc_phy_byte_lane.v`	One complete byte: 8×dq_bit_lane + dqs_bit_lane + dm_bit_lane
`ddr3mc_phy_dq_bit_lane.v`	One DQ bit: IOBUF + IDELAY + ODELAY + IDDRE1 + ODDRE1
`ddr3mc_phy_dqs_bit_lane.v`	One DQS bit: differential IO + IDELAY + ODELAY + DDR registers

File	Purpose
`ddr3mc_phy_dm_bit_lane.v`	One DM bit: output only + ODELAY for alignment
`ddr3mc_phy_data_lane.v`	Data lane aggregation helper
`ddr3mc_phy_ck_generator.v`	DDR3 clock: ODDRE1 + ODELAY + differential output pad
`ddr3mc_phy_control_lanes.v`	Addr/cmd: registers all DDR3 control signals through OBUF cells
`ddr3mc_phy_pvt_cal.v`	PVT impedance calibration: ZQ cell + cal.v controller

DDR3MC/rtl/ — Custom ASIC Cells

File	Purpose
`IGIDDRT01A.v`	Complete GUC DDR IO cell library (IOBUF, OBUF, IBUF, power pads)
`DLL_DCC_v003.v`	DLL with Duty Cycle Correction: 1 master + up to 5 slaves
`DLL_TOP_S1.v`	Single-bit DLL wrapper (for DQS, CK lanes)
`DLL_TOP_S4.v`	4-bit DLL wrapper (for DQ groups)
`IGADLLT04A_MASTER_v003.v`	DLL master: measures clock period, outputs LEAD/LAG
`IGADLLT04A_SLAVE_v003.v`	DLL slave: programmable delay line, 512 tap positions
`IGADLLT04A_MATCH_v003.v`	DLL match cell: equalizes routing delays
`IGADLLT04A_PD_v003.v`	DLL phase detector: drives feedback convergence
`DELAY_S1.v`	Single-bit delay cell model
`DELAY_S4.v`	4-bit parallel delay cell model
`iddre1.v`	Input DDR flip-flop: captures on both clock edges
`oddre1.v`	Output DDR flip-flop: drives on both clock edges
`cal.v`	ZQ/PVT calibration algorithm: sweeps pu/pd codes using pcomp/ncomp

DDR3MC/rtl/ — Wrappers & Test

File	Purpose
`ddr3mc_fpga_wrap.sv`	FPGA validation wrapper with Xilinx clocks and IOs
`ddr3mc_test_driver.sv`	Automated R/W test pattern generator for FPGA hardware validation
`header.sh` / `header.txt`	Copyright header auto-insertion utility

DDR3MC/sim/

File	Purpose
`ddr3.sv`	Full DDR3 SDRAM behavioral model (timing checks, memory array, all commands)
`ddr3_sim_sodimm_x8_2R.sv`	2-rank SO-DIMM model (multiple ddr3.sv instances)
`spd_eeprom.sv`	SPD EEPROM I2C slave model (serves eeprom.txt contents)
`eeprom.txt`	256 bytes ofreal DDR3-1600 SPD data
`24AA02.v`	Microchip 24AA02 EEPROM vendor model
`ddr3mc_tb_top.sv`	Primary testbench: DUT + DDR3 model + SPD + clock + reset + stimulus
`tb_ddr3mc_top.sv`	Alternative testbench with more test scenarios
`tb_phy_top.sv`	PHY isolation testbench
`ddr3mc_sim_bfm.sv`	Bus functional model with mem_write/mem_read tasks
`4096Mb_ddr3_parameters.svh`	JEDEC 4Gb DDR3 timing constants for ddr3.sv
`8192Mb_ddr3_parameters.svh`	JEDEC 8Gb DDR3 timing constants for ddr3.sv

MemCtrl/

File	Purpose
`memory_controller_top.sv`	System top: CPU bus + DDR3MC + SFC + SPDC + I2C + SPI + TMB
`memory_controller_interface_to_ddr3mc.v`	Protocol bridge: decoupled bus ↔ DDR3MC direct interface + scrambler
`RRArbiter.v`	Round-robin arbiter for 2 CPU ports (Chisel-generated)
`Queue.v` (QueueCompatibility)	Response ordering queue (Chisel-generated)
`memory_controller_fpga_wrap.sv`	FPGA validation wrapper
`memory_controller_fpga_test_driver.sv`	FPGA automated test stimulus
`memory_controller_tmb.sv`	Test Memory Bus PCIe interface for board testing
`memory_controller_top.v` (rtlasic/)	ASIC-specific version oftop module

Utility Modules (MemCtrl1/src/import/)

File	Purpose
`scrambler.v`	512-bit LFSR data scrambler (3-bit LFSR, polynomial x^3+x+1)
`pipeline.v`	Configurable-depth pipeline register
`reset_sync.v`	Two-flop asynchronous reset synchronizer

17. KEY DESIGN NUMBERS SUMMARY

Parameter	Value	Notes
Clock frequency	200 MHz	Halfdata rate
DDR3 data rate	DDR3-400 (400 MT/s)	200 MHz × 2 edges

Parameter	Value	Notes
System bus width	512 bits (64 bytes)	One full DDR3 burst
DDR3 DQ width	64 bits (8 bytes)	8 byte lanes
System address width	33 bits	8 GB addressable
DIMMs supported	1 (up to 3 in MemCtrl)	DIMM0 connected
Ranks per DIMM	2	rank0 and rank1
Row address bits	16	64K rows
Column address bits	11	2K columns
Bank address bits	3	8 banks
CMDQ depth	4	Outstanding commands
WDQ/RDQ depth	4	Write/read data entries
Delay tap resolution	~10ps per tap (512 taps / 5000ps)	DLL slave
Calibration steps	6 sequential	Must all pass
DLL slave instances	Per DQ group (×4), per DQS (×1), per CK (×1)	Multiple DLLs
CAS latency tCL	6 cycles	At 200 MHz = 30ns
CAS write latency tCWL	5 cycles	At 200 MHz = 25ns
Refresh interval tREFI	1560 cycles	7.8μs at 200 MHz
ZQ calibration	512 cycles (tZQCL)	Periodic recalibration
Custom ASIC cell library	GUC (Global Unichip / TSMC)	IGIDDRT01A + IGADLLT04A

18. STATUS BIT DEFINITIONS (sys_status[7:0])

From the source code:

- `STATUS_BIT_RW_READY` – set when controller enters normal operation

- `STATUS_BIT_CAL_DONE` —set when all 6 calibration steps complete
- `STATUS_BIT_INIT_DONE` – set when DDR3 initialization complete
- `STATUS_BIT_CFG_DONE` —set when configuration loaded
- `STATUS_BIT_CFG_ERR` —set ifunsupported DIMM configuration detected
- `STATUS_BIT_CAL_ERR` —set ifcalibration fails (future: commented out)
